Analiza algorytmów

Maciej Pluta, 287339

1 Wprowadzenie

Projekt obejmował implementację i szczegółową analizę czterech fundamentalnych algorytmów. Każdy algorytm został zaimplementowany w kilku wariantach, co pozwoliło na porównanie ich wydajności, poprawności oraz praktycznej użyteczności w różnych scenariuszach.

Przeanalizowane algorytmy:

- **CUT-ROD** – algorytm optymalizacji cięcia pręta wykorzystujący programowanie dynamiczne
- **LCS (Longest Common Subsequence)** – algorytm znajdowania najdłuższego wspólnego podciągu
- **ACTIVITY SELECTOR** – algorytm wyboru maksymalnego zbioru niekonfliktujących zajęć
- **HUFFMAN** – algorytm konstrukcji optymalnego kodu prefiksowego (wersja ternarna)

2 Kluczowe fragmenty implementacji

2.1 CUT-ROD

Algorytm optymalizacji cięcia pręta na kawałki w celu maksymalizacji zysku. Mając pręt długości n i tablicę cen $p[i]$ za pręt długości i, algorytm znajduje optymalny podział. Kluczową cechą jest optymalna podstruktura.

```
int cutRodBottomUp(const vector<int>& p, int n, vector<int>& s) {
    vector<int> dp(n+1, 0);
    for (int j = 1; j <= n; j++) {
        int best = INT_MIN;
        for (int i = 1; i <= j; i++) {
            if (p[i-1] + dp[j-i] > best) {
                best = p[i-1] + dp[j-i];
                s[j] = i;
            }
        }
        dp[j] = best;
    }
    return dp[n];
}
```

Listing 1: CUT-ROD – wersja Bottom-Up

Działanie: Algorytm wypełnia tablicę `dp` od najmniejszych do największych długości. Dla każdej długości j sprawdza wszystkie możliwe pierwsze cięcia i, wybierając to, które daje maksymalny zysk.

Złożoność: $O(n^2)$ czasowa, $O(n)$ pamięciowa

2.2 LCS

Algorytm znajduje najdłuższy podciąg wspólny dla dwóch sekwencji przy wykorzystaniu tablicy dwuwymiarowej. Zaimplementowano dwie wersje: rekurencyjną ze spamiętywaniem i iteracyjną.

```
string lcsIter(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1,0));

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            if (a[i-1] == b[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

    string res;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (a[i-1] == b[j-1]) {
            res = a[i-1] + res;
            i--; j--;
        } else if (dp[i-1][j] >= dp[i][j-1]) i--;
        else j--;
    }
    return res;
}
```

Listing 2: LCS – wersja iteracyjna

Działanie: Tablica `dp[i][j]` przechowuje długość LCS dla prefiksów. Jeśli znaki są równe, długość zwiększa się o 1. W przeciwnym razie wybierane jest lepsze z rozwiązań dla krótszych prefiksów.

Złożoność: $O(n \cdot m)$ czasowa, $O(n \cdot m)$ pamięciowa

2.3 ACTIVITY SELECTOR

Problem wyboru zajęć polega na znalezieniu maksymalnego zbioru niekonfliktujących aktywności. Algorytm zachłanny sortuje zajęcia po czasie zakończenia i wybiera kolejne dostępne zajęcia. Strategia ta gwarantuje optymalność dzięki własności zachłannego wyboru.

```
vector<pair<int,int>> activityIterative(vector<pair<int,int>> a) {
    sort(a.begin(), a.end(), [](auto& x, auto& y) {
        return x.second < y.second;
    });

    vector<pair<int,int>> res;
    int last = 0;
    for (auto& x : a) {
        if (x.first >= last) {
            res.push_back(x);
```

```
            last = x.second;
        }
    }
    return res;
}
```

Listing 3: ACTIVITY SELECTOR – wersja zachłanna

Działanie: Algorytm sortuje zajęcia rosnąco po czasie zakończenia. Następnie iteracyjnie wybiera każde zajęcie, które zaczyna się po zakończeniu poprzednio wybranego. Zawsze wybiera zajęcie kończące się najwcześniej spośród dostępnych.

Złożoność: $O(n \log n)$ czasowa (dominowane przez sortowanie), $O(n)$ pamięciowa

2.4 HUFFMAN

Algorytm konstruuje optymalny kod prefiksowy minimalizujący średnią długość zakodowanej wiadomości. Wersja ternarna używa trzech symboli (0,1,2) zamiast dwóch standardowych. Algorytm buduje drzewa łączenia węzłów o najmniejszych częstotliwościach.

```
map<char,string> huffman(const map<char,long long>& freq) {
    priority_queue<Node*, vector<Node*>, Cmp> pq;
    for (auto& x : freq)
        pq.push(new Node(x.second, x.first));

    const int k = 3;
    while (pq.size() > 1) {
        int take = min(k, (int)pq.size());
        Node* n = new Node(0);
        for (int i = 0; i < take; i++) {
            auto t = pq.top(); pq.pop();
            n->ch.push_back(t);
            n->f += t->f;
        }
        pq.push(n);
    }

    map<char,string> res;
    Node* root = pq.top();
    gen(root, "", res);
    del(root);
    return res;
}
```

Listing 4: HUFFMAN – wersja ternarna

Działanie: Kolejka priorytetowa przechowuje węzły posortowane rosnąco po częstotliwościach. W każdej iteracji łączone są 3 węzły o najmniejszych częstotliwościach. Nowy węzeł otrzymuje sumę częstotliwości dzieci. Proces kończy się gdy w kolejce pozostanie tylko korzeń drzewa.

Złożoność: $O(n \log n)$ czasowa, $O(n)$ pamięciowa

3 Wyniki testów i analiza

3.1 Testy poprawności

Tabela 1: Wyniki testów poprawności

Algorytm / Test	**Wynik**
CUT-ROD (n=4)	Naive: 10, Memo: 10 | 2 2, Bottom-Up: 10 | 2 2
LCS	"BCBA"(obie implementacje)
ACTIVITY SELECTOR	(1,4) (5,7) (8,11)
HUFFMAN	a: 120, b: 121, c: 122, d: 10, e: 11, f: 0

Komentarz: Wszystkie implementacje każdego algorytmu zwracają identyczne wyniki, co potwierdza ich poprawność. Dla CUT-ROD wszystkie trzy wersje zwracają maksymalny zysk 10, a wersje Memo i Bottom-Up dodatkowo odtwarzają optymalny podział 2+2.

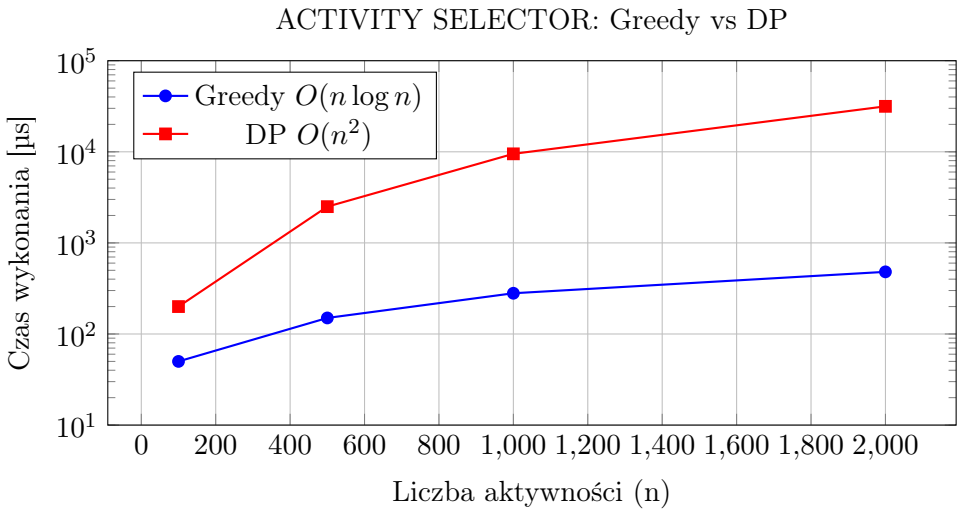
3.2 Testy wydajnościowe

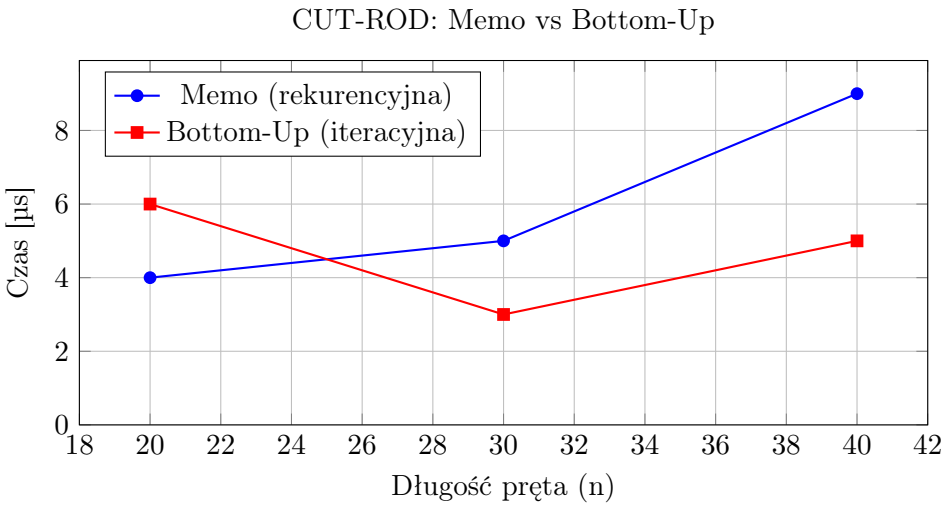
Tabela 2: Czas wykonania w mikrosekundach [µs]

Algorytm	**Wersja 1**	**Wersja 2**	**Komentarz**
CUT-ROD (n=20)	Memo: 4	Bottom-Up: 6	Wersja rekurencyjna ze spamiętywaniem jest szybsza
CUT-ROD (n=30)	Memo: 5	Bottom-Up: 3	Wersja iteracyjna zaczyna być szybsza
CUT-ROD (n=40)	Memo: 9	Bottom-Up: 5	Różnica na korzyść Bottom-Up staje się wyraźna
ACTIVITY (2000)	Greedy: 481	DP: 31554	Algorytm zachłanny jest 65× szybszy
LCS (500x500)	Iteracyjny: 3000	—	Dobry czas dla dużych danych

Komentarz ogólny: Testy wydajnościowe pokazują wyraźną przewagę wersji iteracyjnych i algorytmów zachłannych dla dużych zbiorów danych. Algorytm naiwny CUT-ROD nie był testowany dla n>10 ze względu na złożoność wykładniczą.

4 Wizualizacja wyników





5 Wnioski

CUT-ROD

Zalety: Gwarantuje rozwiązanie optymalne, prosty w implementacji, łatwe odtwarzanie rozwiązania.
Wady: Złożoność kwadratowa $O(n^2)$, wersja naiwna ma złożoność wykładniczą.
Zastosowanie kiedy: Dla małych i średnich n, gdy potrzebne jest dokładne optymalne rozwiązanie.
Wersja preferowana: Bottom-Up dla n $>$ 25, Memo dla mniejszych danych.

LCS

Zalety: Gwarantuje optymalne rozwiązanie, dobrze skalowalny dla umiarkowanych danych.
Wady: Wysokie zużycie pamięci $O(n \cdot m)$, wolniejszy dla bardzo długich sekwencji.
Zastosowanie kiedy: Porównywanie sekwencji tekstowych lub genetycznych o umiarkowanej długości.
Wersja preferowana: Iteracyjna - bardziej stabilna wydajnościowo.

ACTIVITY SELECTOR

Zalety: Algorytm zachłanny jest bardzo szybki $O(n \log n)$, prosty w implementacji.
Wady: DP ma złożoność $O(n^2)$, co czyni go niepraktycznym dla dużych danych.
Zastosowanie kiedy: Zawsze preferować wersję zachłanną - jest optymalna i znacznie szybsza.
Różnica wydajności: Greedy 65× szybszy niż DP dla 2000 zajęć.

HUFFMAN

Zalety: Generuje optymalne kody prefiksowe, wersja ternarna działa poprawnie.
Wady: Konieczność przechowywania drzewa kodowego, koszt kodowania/odkodowania.
Zastosowanie kiedy: Kompresja danych, gdy częstotliwości symboli są znane i nierównomierne.

Ogólne wnioski

1. **Iteracyjne vs rekurencyjne:** Wersje iteracyjne (Bottom-Up) są szybsze dla dużych danych dzięki brakowi narzutu rekurencyjnego.

2. **Zachłanne vs dynamiczne:** Algorytmy zachłanne są znacznie szybsze gdy problem ma własność zachłannego wyboru (ACTIVITY SELECTOR 65× szybszy).

3. **Skalowalność:** CUT-ROD $O(n^2)$ słabo skalowalny, LCS $O(n \cdot m)$ umiarkowanie, Greedy ACTIVITY $O(n \log n)$ dobrze.

4. **Praktyczność:** Wersje naiwne (rekurencyjne bez optymalizacji) są niepraktyczne nawet dla średnich danych.

5. **Wybór algorytmu:** Zależy od rozmiaru danych, wymagań wydajnościowych i struktury problemu.

Podsumowanie: Projekt potwierdził, że nie ma uniwersalnie najlepszego algorytmu. Wybór między programowaniem dynamicznym a podejściem zachłannym zależy od specyfiki problemu. Dla problemów z optymalną podstrukturą i nakładającymi się podproblemami (CUT-ROD, LCS) DP jest konieczne. Dla problemów z własnością zachłannego wyboru (ACTIVITY SELECTOR, HUFFMAN) algorytmy zachłanne są znacznie efektywniejsze.